

Web API da MecâniQA: Engenharia de Software Aplicada

Autores: Alex Gabriel Alves dos Santos
Gustavo Azevedo Diniz
Gustavo Muniz dos Anjos
João Paulo Rosa Batista
Matheus Silva do Monte

Agenda

1. O Problema da Oficina:

Orquestração do atendimento, integridade dos dados e preço histórico.

2. Padrão Builder:

Construtor privado e criação fluente e segura da OrdemServico.

3. Entidade Associativa:

ItemPedidoPecas com peças repetidas e preço congelado.

4. Padrão Singleton:

Repositório em memória compartilhado com o Controller.

5. Diagrama de Atividades (US03):

Fluxo do POST /api/pedidos com validações e respostas HTTP.

6. DTOs e Mappers:

Contratos entre camadas sem expor entidades.

7. Diagramas e endpoints:

Classes, relacionamentos atividades e CRUD

O objetivo desta apresentação é mostrar como o MVP da Fase 1 resolve os desafios reais da MecâniQA com engenharia orientada a objetos.

O Problema da Oficina



- **Fluxo do MVP da Fase 1**

- Cadastro de peças e serviços evolui para o atendimento completo: Ordem de Serviço, Pedido de Peças e atualização de status.

- **Integridade do domínio**

- Builder e validações evitam objetos incompletos; DTOs e Mappers protegem as fronteiras da API.

- **Preço histórico**

- ItemPedidoPecas registra quantidade e precoUnitario no momento da inclusão, sem sofrer com reajustes futuros do catálogo.

Padrão Builder

- Múltiplos construtores oferecem formas controladas de criação
- this diferencia atributo e parâmetro e mantém o encadeamento do Builder
- Construtor de OrdemServico é privado: new OrdemServico() não compila
- Criação fluente termina em build()
- build() valida cliente e serviços
- Teste automatizado garante a blindagem da instanciação

```
OrdemServico.java

public class OrdemServico {
    // Construtor ESTRITAMENTE PRIVADO: impede new OrdemServico()
    private OrdemServico(Builder builder) {
        this.codigo = builder.codigo;
        this.cliente = builder.cliente;
        this.veiculo = builder.veiculo;
        this.status = builder.status != null ? builder.status : StatusOrdemServico.ABERTO;
        this.servicos = builder.servicos != null ? builder.servicos : new ArrayList<>();
        this.pedidoPecas = builder.pedidoPecas;
        // Total financeiro calculado de forma imutável
        this.valorTotal = calcularValorTotal(this.servicos, this.pedidoPecas);
        this.dataAbertura = LocalDateTime.now();
    }

    public static Builder builder() {
        return new Builder();
    }
}
```

Entidade Associativa

- PedidoPecas 1 — 1..*
ItemPedidoPecas
- ItemPedidoPecas é a entidade associativa entre PedidoPecas e Peca
- A mesma peça pode aparecer em itens diferentes
- quantidade e precoUnitario são atributos do relacionamento
- O preço é congelado no momento da inclusão
- Subtotal usa BigDecimal, sem erro de arredondamento

```
ItemPedidoPecas.java

public class ItemPedidoPecas {
    private Long codigo;
    private Peca peca;
    private Integer quantidade;
    private BigDecimal precoUnitario;

    // Subtotal calculado com precisão monetária (BigDecimal)
    public BigDecimal getSubtotal() {
        if (precoUnitario == null || quantidade == null) return BigDecimal.ZERO;
        return precoUnitario.multiply(BigDecimal.valueOf(quantidade));
    }

    public static class Builder {
        //REGRA DE NEGÓCIO: Congela o preço no momento da inclusão
        public Builder peca(Peca peca) {
            this.pecas.add(pecas);
            if (pecas != null && this.precoUnitario == null) {
                this.precoUnitario = peca.getPrecoVenda(); // Preço histórico
            }
            return this;
        }
    }
}
```

Padrão Singleton

- Persistência em listas na memória, sem banco de dados
- Instância única: os dados não somem a cada requisição HTTP
- UML: estereótipo <<Singleton>> e membros estáticos sublinhados
- Controller usa getInstance(), sem new

```
PecaRepository.java & Controller

public class PecaRepository {
    // 1. Atributo estático guarda a instância única
    private static PecaRepository instance;
    private List<Peca> pecas;

    //2.Construtor PRIVADO impede 'new PecaRepository()' externo
    private PecaRepository() {
        this.pecas = new ArrayList<>();
    }

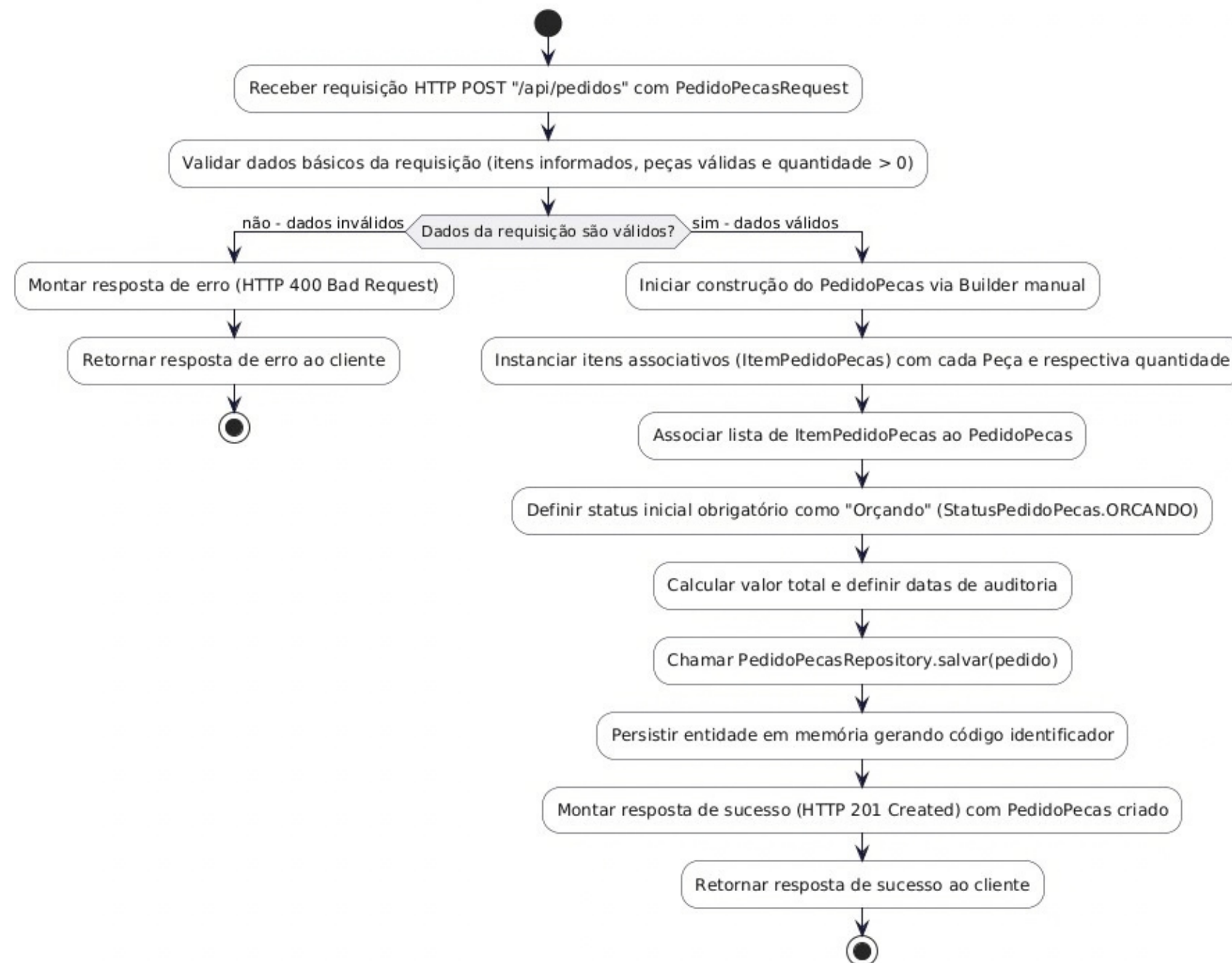
    //3.Método estático global de recuperação da instância única
    public static PecaRepository getInstance() {
        if(instance == null) {
            instance = new PecaRepository();
        }
        return instance;
    }
}

@RestController
public class PecaController {
    private PecaRepository pecaRepository;

    //injeção limpa sem frameworks usando getInstance()
    public PecaController() {
        this.pecasRepository = PecaRepository.getInstance();
    }
}
```

Diagrama de Atividades (US03)

US03 - Criar Pedido de Peças - Diagrama de Atividades (Visão do Controller)



- Recebe POST /api/pedidos com PedidoPecasRequest
- Valida itens, peças e quantidade > 0
- Dados inválidos: HTTP 400 Bad Request
- Dados válidos: itens e pedido via Builder, status ORCANDO
- Calcula total e salva no repositório Singleton
- Responde HTTP 201 Created

DTOs e Mappers



Controllers recebem DTOs e devolvem DTOs

OrdemServicoRequest e StatusOrdemServicoRequest isolam as entradas da OS

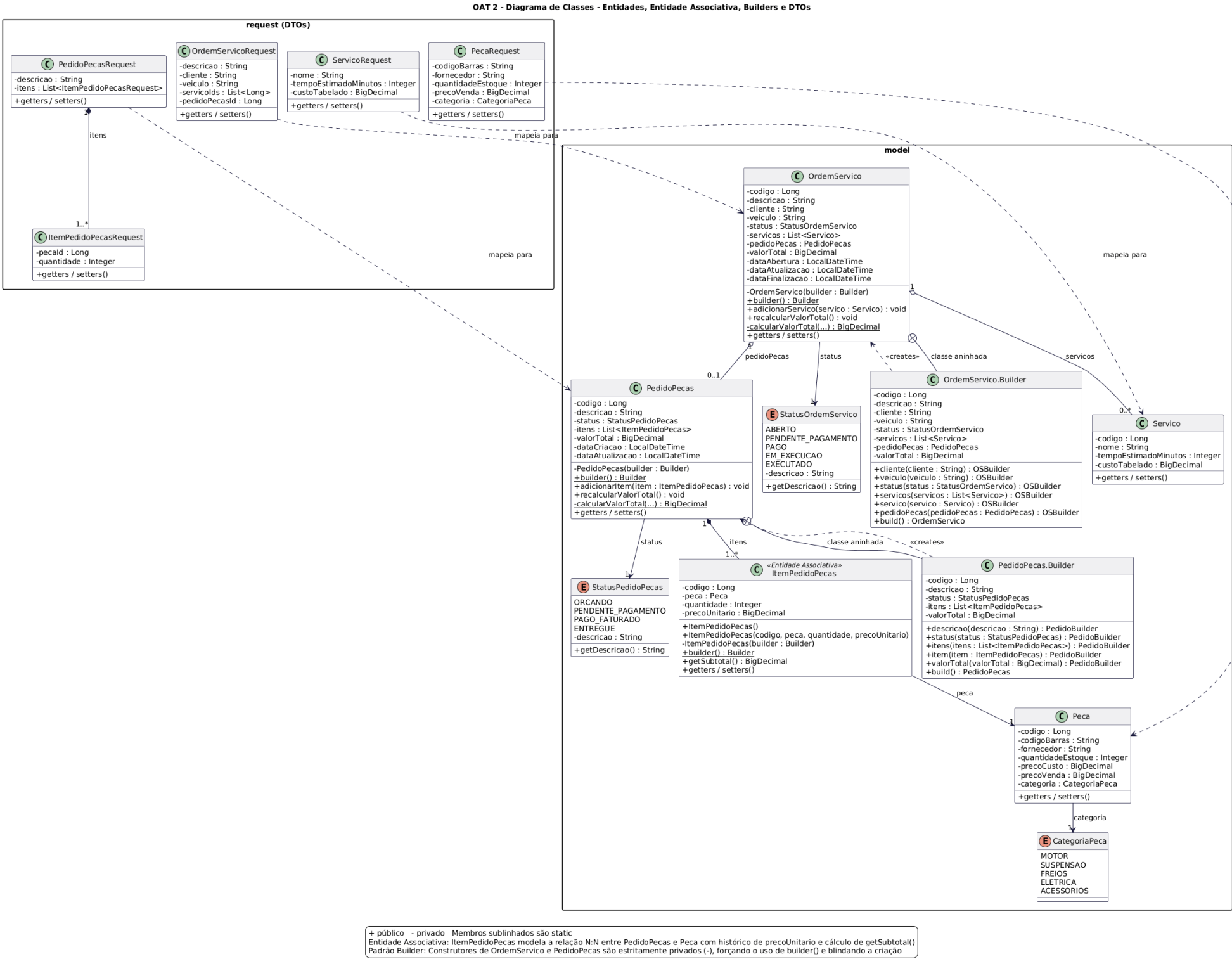
PedidoPecasRequest e ItemPedidoPecasRequest isolam criação e inclusão de peças

Mappers fazem o de-para DTO ↔ entidade em classes utilitárias

Entidades não são expostas diretamente na API

Validações de quantidade, referência e status ficam no Controller

Diagrama de Classes



Relacionamentos e Multiplicidades



OrdemServico 1 o-- 0..* Servico: agregação do conjunto de serviços

OrdemServico 1 o-- 0..1 PedidoPecas: pedido associado à OS

PedidoPecas 1 *-- 1..* ItemPedidoPecas: composição do ciclo de vida dos itens

ItemPedidoPecas * --> 1 Peca: referência unidirecional ao catálogo

Atributos do relacionamento: quantidade e precoUnitario congelado

Dependências: Controller → DTO/Mapper/Repository; Builder → entidade

CRUD da Fase 1



US01 POST /api/ordens-servico → 201 Created
US02 PUT /api/ordens-servico/{codigo}/status → 200 OK
US03 POST /api/pedidos → 201 Created
US04 POST /api/pedidos/{codigo}/itens → 200 OK
US05 PUT /api/pedidos/{codigo}/status → 200 OK
Erros tratados: dados inválidos → 400; código inexistente → 404

Considerações finais



Problema resolvido: atendimento de ponta a ponta com OS e Pedido de Peças

Encapsulamento: Builder manual, construtores privados e Singleton em memória

Contratos limpos: Controllers usam DTOs e Mappers, sem expor entidades

Rastreabilidade: diagramas de classes e atividades refletem os endpoints implementados

Referências



- Repositório: Guztaver/mecaniQA-api-rio-de-janeiro
- Caderno de Guias — OAT 2 — Rio de Janeiro
- OAT 1 — Web API Fundacional da MecâniQA
- Modelo de Apresentação Institucional UNEX
- Testes: OrdemServicoTest, PedidoPecasTest e testes dos endpoints US01–US05